

Let’s Scale Step by Step: Compute-Efficient Hyperparameter Transfer for Large-Scale Mixture-of-Experts

Nayeon Kim^{1*}, Hojin Lee^{2†}, Yunju Bak¹, Jaesun Park¹, Boseop Kim^{1*}

¹Kakao Corp., ²Upstage AI

Abstract

Mixture-of-Experts (MoE) architectures significantly expand model capacity without a proportional increase in computational cost. However, optimizing their hyperparameters—particularly the learning rate—at extreme scales of both model size and token budget via sweeping remains computationally prohibitive. In this paper, we propose a compute-efficient, two-step hyperparameter transfer framework that estimates optimal learning rates for training large MoE models by transferring them across scaling model widths, and subsequently extrapolating to trillion-token horizons. First, we formulate a Maximal Update Parameterization (μ P) adaptation for MoE architectures utilizing Multi-head Latent Attention (MLA) and the Muon optimizer, demonstrating that optimal learning rates transfer consistently across width-scaled models. Second, we extend this transferability along the token dimension by establishing a predictive scaling law. By applying linear regression to the optimal values derived from small proxy models on limited budgets, we successfully extrapolate the ideal learning rate to massive training horizons (e.g., 10 trillion tokens) with high fidelity ($R^2 = 0.95$). Consequently, this indicates that proxy training on small models is sufficient to determine the optimal learning rate for the extensive training of large-scale MoEs. We apply the proposed methodology to pretrain our foundation model (155B total, 17B active parameters) from scratch, and the stable training and evaluation results validate that optimal configurations for full-scale target models can be accurately predicted with minimal ablation costs.

1 Introduction

Mixture-of-Experts (MoE) architectures (Shazeer et al., 2017) have emerged as a powerful paradigm for scaling LLM capacity without a proportional increase in computational cost, enabling both greater model specialization and cost-efficient inference. As a result, MoE models can achieve significantly higher effective capacity than dense counterparts trained on the same data, delivering stronger performance without any additional training cost (Shazeer et al., 2017; Artetxe et al., 2021; Fedus et al., 2022; Ludziewski et al., 2024). Prominent open-source LLMs such as DeepSeek-V3 (Liu et al., 2024b), Qwen3-235B-A22B (Yang et al., 2025), Kimi-K2.5 (Team et al., 2026), and GLM-5 (Zeng et al., 2026) employ MoE designs to deliver powerful performance while reducing the per-token active parameters and active inference FLOPs, often outperforming dense counterparts of similar or larger size on a range of tasks. However, MoE architectures introduce additional hyperparameters to manage expert routing and load balancing, which expands the hyperparameter space and makes tuning considerably more complex and expensive than for dense models. Among these hyperparameters, the learning rate stands out as a critical determinant of successful training (Bengio, 2012; Smith, 2018; Kaplan et al., 2020; Yang et al., 2022; Li et al., 2025b); yet its optimal value is highly sensitive to both model size and token budget (Li et al., 2025b; Bjorck et al., 2025). This means that every change in scale—whether in model size or training horizon—demands a fresh and expensive hyperparameter search.

*Contact: {lana.ny, mat.mul}@kakaocorp.com

†Work done at Kakao Corp.

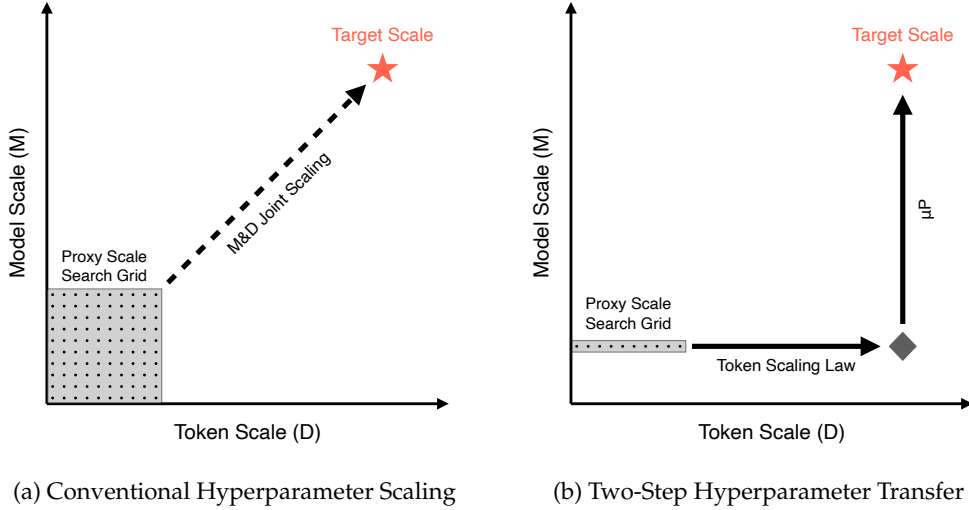


Figure 1: (a) In conventional hyperparameter optimization, exhaustive 2D sweeps across both model scale (M) and token scale (D) are required to jointly predict the target scale ($\star$), incurring prohibitive computational costs. (b) Our approach decouples the two dimensions: μP -based width transferability eliminates model-scale sweeps, while a token scaling law from a few proxy runs enables direct extrapolation to the target scale ($\star$), replacing 2D sweeps with a lightweight 1D search along the token dimension.

To mitigate the prohibitive computational cost of hyperparameter sweeps at massive scales, frameworks such as Maximal Update Parameterization (μP) (Yang & Hu, 2021) and μ -Transfer (Yang et al., 2022) have been proposed. These methods enable the zero-shot transfer of optimal hyperparameters (e.g., learning rates) from small proxy models to larger variants. However, these methods were originally designed for dense models, where scaling width is the primary axis. MoE architectures introduce an additional scaling dimension—sparsity—governed by the ratio of active to total experts. For MoE models at extreme parameter scales (e.g., over 100B total parameters), simply expanding the width (e.g., hidden size) becomes increasingly intractable due to prohibitive inference costs and hardware constraints. Instead, increasing the total number of experts while keeping active experts fixed offers a more practical path, as it expands model capacity without a proportional increase in inference compute. Nevertheless, sparsity follows distinct scaling behaviors (Clark et al., 2022; Tian et al., 2025a; Team et al., 2025), and it remains unclear whether existing hyperparameter transfer frameworks derived for width scaling generalize to the sparsity dimension. This critical gap remains largely unaddressed, leaving optimal hyperparameter configuration prediction for large-scale MoE training an open problem.

To bridge this gap, we propose a practical, resource-efficient two-step hyperparameter transfer framework for large-scale MoE pretraining. Figure 1 illustrates the difference between conventional hyperparameter scaling approaches and our proposed method. Conventional approaches consider model and token scales simultaneously, thereby relying on exhaustive

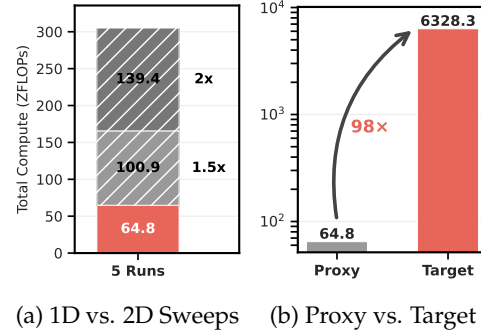


Figure 2: **Proxy** denotes the 5 proxy-scale runs from Section 3.3.2 and **Target** denotes our large-scale training from Section 3.3.3. (a) 2D sweeps require scaling across model size, resulting in additional computation. Model scale search (1.5 $\times$, 2 $\times$ wider than **Proxy**; details in Appendix B) incurs an additional 240.3 ZFLOPs of compute beyond the **Proxy** run cost of 64.8 ZFLOPs. (b) Total computation of the **Target** is approximately 98 $\times$ larger than the sum of the **Proxy** runs. FLOPs are computed following Narayanan et al. (2021).

2D sweeps across both dimensions to empirically predict optimal configurations at large scale. In contrast, our approach leverages μP to enable robust transfer across model scales. This requires only a few small-scale proxy runs to extrapolate the optimal learning rate along the token dimension, allowing for a direct transfer to full-scale model training. Figure 2a illustrates how these 2D sweeps incur computation overhead as the model scale search space grows. Our findings suggest that it is entirely feasible to estimate optimal training hyperparameters (i.e., learning rates) for extreme scales—spanning both massive model widths and long token horizons—without resorting to costly trial-and-error sweeps. We validate our methodology by pretraining our foundation MoE model (155B total, 17B active parameters) over 10 trillion tokens from scratch. Details are summarized in Section 3.3.3.

Our main contributions are as follows:

- **μP Adaptation for MoE Architectures:** Building on prior work discussing the adaptation of μP to MoE architectures, we study zero-shot hyperparameter transfer when scaling both model width and the total number of experts (i.e., sparsity expansion). We further explore the use of the Muon optimizer in this setting, extending the empirical scope of μP -based MoE scaling.
- **Token-Scale Extrapolation within a Two-Step Framework:** We introduce a two-step predictive framework to extrapolate optimal learning rates to unseen, long token horizons based on cost-effective small-scale proxy runs, significantly reducing the computational overhead of hyperparameter tuning.
- **Large-Scale Validation:** We apply the proposed two-step framework to pretrain a large-scale MoE foundation model (155B total, 17B active parameters) from scratch over 10 trillion tokens, validating its practical effectiveness at scale.

2 Methods

Our two-step hyperparameter transfer framework requires two key components: (1) formulating μP specifically for MoE architectures, and (2) establishing a scaling law that predicts the optimal learning rate as the token budget scales. Section 2.1 describes our adaptation of μP to MoE architectures, and the results in Section 3.2 validate that this formulation ensures that the optimal learning rate can be reliably transferred across model widths. Section 2.2 then addresses extrapolation to extended training horizons: Section 2.2.1 defines the optimal learning rate at a given token scale, and Section 2.2.2 presents our methodology for extending these estimates to unseen, large-scale token horizons. The efficacy of this extrapolation is demonstrated in Section 3.3.

2.1 Applying μ -Parameterization to MoE

Although determining the optimal learning rate is critical for successful model training, conducting comprehensive hyperparameter search for large-scale MoE models exceeding 100B parameters is computationally prohibitive. Therefore, we adopt Maximal Update Parameterization (μP) (Yang & Hu, 2021) that offers a potential solution by enabling zero-shot hyperparameter transfer. Following μ -Transfer (Yang et al., 2022), Table 1 categorizes model parameters based on their expansion behavior under width scaling. Since none of our parameters have shapes invariant to model width (i.e., tied to fixed constants such as vocabulary size or context length), we exclude scalar-like parameters. Vector-like parameters receive μP initialization only, while matrix-like parameters are subject to both μP initialization and learning rate scaling, as detailed in Table 2.

Following the findings of Wortsman et al. (2024), which show that the properties of μP can be maintained by solely scaling the learning rates for linear layers, we apply learning rate scaling exclusively to matrix-like hidden weights. As depth scaling is known to be highly unstable for μP -based hyperparameter transfer (Yang et al., 2022; Wortsman et al., 2024), we consider only width scaling while keeping the depth (i.e., number of layers) fixed. For attention, the head dimension remains fixed while the number of heads scales proportionally with the hidden dimension (Yang et al., 2022; Wortsman et al., 2024).

Parameter Type	Definition	Examples
Vector-like	Parameters that contain exactly one infinitely expandable dimension.	Embeddings, biases, expert FC2 weights
Matrix-like	Parameters that contain two infinitely expandable dimensions.	FFN, attention, router, expert FC1 weights

Table 1: Parameter classification by shape invariance to model width. For MoE layers, router parameters and expert FC1 weights are classified as matrix-like parameters, while expert FC2 weights are treated as vector-like since their effective input dimensionality remains bounded by the fixed number of active experts and the fixed MoE intermediate dimension.

	I/O Embeddings & All Biases (Vector-like)	Hidden Weights (Vector-like)	Hidden Weights (Matrix-like)
Init. Var	0.04	$\text{fan_in}_{\text{base}} / \text{fan_in}$	$\text{fan_in}_{\text{base}} / \text{fan_in}$
LR Scaling Factor	1	1	$\text{fan_in}_{\text{base}} / \text{fan_in}$

Table 2: μP and learning rate scaling factors for dense and MoE models. fan_in and $\text{fan_in}_{\text{base}}$ are the fan-in values of the width-scaled model and the base proxy model, respectively. Learning rate scaling factors are multiplied by the learning rate when updating the corresponding parameters. Notably, for MLA, the low-rank projection dimensions for query and key-value are kept fixed during width scaling. Since these serve as the fan_in of the corresponding up-projection matrices, the learning rate scaling applied to these matrices has no effect, as the scaling factor reduces to 1.

When scaling up the MoE models, we fix the number of active experts per token and the MoE intermediate dimension, while increasing the total number of experts and the hidden dimension. This reflects a practical MoE scaling strategy: rather than expanding width alone, increasing sparsity raises model capacity while keeping per-token inference costs minimal (Team et al., 2025). Consequently, sparsity becomes a key scaling axis alongside width for μP -based hyperparameter transfer. Under this scaling path, total scale, active scale, and sparsity are coupled rather than treated as independent axes. This coupling is consistent with the spectral-condition view of μP : changing the active ratio by increasing the number of experts does not introduce any additional change to the fan-in or fan-out of each individual expert beyond that induced by width scaling, and therefore remains compatible with the same μP scaling rule (Yang et al., 2023). This setting is further motivated by practical efficiency considerations. Recent MoE scaling strategies increasingly rely on lower active ratios to expand model capacity efficiently, but very high sparsity can be hardware-inefficient at proxy scale due to low arithmetic intensity. Running the proxy search at lower sparsity and transferring to a higher-sparsity target therefore keeps the search itself efficient while still targeting the desired sparse regime.

2.2 Extrapolating Optimal Learning Rates to Long Token Horizons

Although μP -based hyperparameter transfer significantly reduces search costs, performing a direct search for the optimal learning rate over trillions of training tokens (e.g., 10T tokens) remains computationally prohibitive even when using small proxy models.

To address this challenge, we aim to identify the optimal learning rate within a constrained token budget and extrapolate the results to the full target token horizon. Building upon the methodology in Section 2.1, the results presented in Section 3.2 verify that optimal learning rates can be successfully transferred when scaling MoE model width. Therefore, if a scaling rule along the token dimension is established, hyperparameter transfer from small proxy models trained on limited token budgets to large-scale models with extensive token budgets can be optimized through a two-step framework.

Importantly, our framework deliberately focuses on learning rate transfer, excluding batch size. This is not merely a simplification—batch size occupies a fundamentally different role: it is not only a modeling hyperparameter but also a system-level variable frequently

adjusted during training to maximize hardware throughput. Moreover, the literature remains divided on how the optimal batch size scales, with conflicting dependencies on training compute (Bi et al., 2024; Tian et al., 2025a) or the token budget alone (Li et al., 2025b). Similar contradictions exist in critical batch size estimations, where assumptions alternate between dependencies solely on the token budget (Zhang et al., 2024) and training loss-driven dynamics (Li et al., 2025a). Given these contradictions and practical constraints, we fix the batch size to maximize GPU efficiency and treat the learning rate as the primary transfer target. By decoupling learning rate optimization from batch size, we propose a learning rate scaling law that remains robust regardless of the batch size chosen to maximize throughput in a given hardware environment.

2.2.1 Estimating Optimal Learning Rates Over Short Token Budgets

To estimate the optimal learning rate for large-scale training, we train small proxy models over short token horizons across a range of learning rates.

We employ the Warmup-Stable-Decay (WSD) scheduler (Hu et al., 2024) as the target long-horizon training setup, matching the number of warmup steps. However, introducing learning rate decay phase for proxy experiments presents two major challenges. First, it is computationally expensive; even if early decay is applied at an intermediate point, each run yields only a single valid data point at its final checkpoint. Consequently, mapping out optimal learning rates across various token budgets would necessitate a large number of independent experiments. Second, performing premature decay during proxy training risks introducing bias into the loss estimates (Li et al., 2025b). To overcome these limitations, we instead terminate proxy runs during the stable phase without decay and apply Exponential Moving Average (EMA) to the model weights. This approach enables us to extract multiple checkpoints at desired token intervals from a single training run, efficiently generating numerous data points representative of a wide range of token budgets. EMA effectively smooths noise in the parameter trajectory and slows parameter updates, thereby resembling the effect of learning rate decay (Morales-Brotons et al., 2024; Baidu-ERNIE-Team, 2025; Tian et al., 2025b; Li et al., 2026). Zhang et al. (2024) demonstrate that constant learning rate with exponential weight averaging (equivalent to EMA in our setting) performs comparably to cosine decay training in large-batch regimes, with the gap widening as the batch size decreases. Our training uses a large global batch size of 32M tokens, making these large-batch findings particularly relevant to our setting. In practice, prior large-scale training works adopt closely related strategies: Liu et al. (2024b) relies on EMA to approximate post-decay model quality, while Olmo et al. (2025) uses checkpoint weight averaging to estimate model performance during pretraining without repeated learning rate annealing.

Let $\theta^{(t)}$ denote the model parameters at step t . The EMA parameters $\theta_{\text{EMA}}^{(t)}$ are updated as follows:

$$\theta_{\text{EMA}}^{(t)} = \alpha \cdot \theta_{\text{EMA}}^{(t-1)} + (1 - \alpha) \cdot \theta^{(t)} \quad (1)$$

We set the smoothing factor to $\alpha = 0.6$, placing substantial weight on recent updates while mitigating the influence of unstable dynamics during the early stages of training. Model checkpoints are updated via EMA at approximately 2B-token intervals, and the resulting checkpoints at every 10B tokens are used for subsequent analysis. Based on the definition of the effective decay window by Baidu-ERNIE-Team (2025), our EMA configuration implies that the most recent 20B tokens maintain more than 1% influence on the merged weights. To identify the optimal learning rate (η^*) for a given token budget, we evaluate the validation loss on a fixed batch across all tested learning rates using these merged checkpoints. Following Bjorck et al. (2025), for each token scale (B), we model the relationship between the validation loss ($\mathcal{L}$) and the log-transformed learning rate ($\log \eta$) using a second-order polynomial:

$$\mathcal{L}(\eta) = a(\log \eta)^2 + b(\log \eta) + c \quad (2)$$

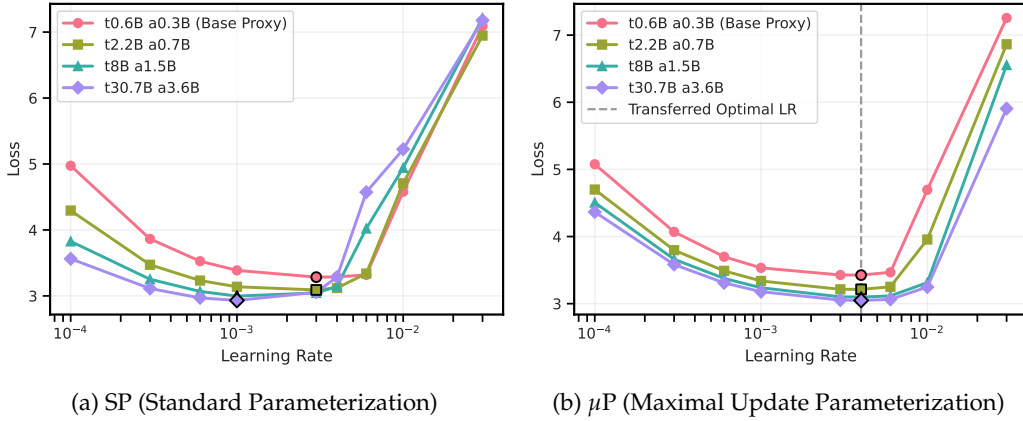


Figure 3: Comparison of learning rate transferability under Standard Parameterization (SP) and Maximal Update Parameterization (μ P) across MLA MoE models of increasing width. Markers with black outlines indicate the learning rate achieving the lowest training loss (i.e., optimal learning rate) for each model. (a) Under SP, the optimal learning rate shifts as model width scales, failing to transfer from the base proxy model (0.6B total, 0.3B active) to larger variants. (b) Under μ P, the optimal learning rate identified in the base proxy consistently transfers across models scaled to $2\times$ (2.2B total, 0.7B active), $4\times$ (8B total, 1.5B active), and $8\times$ (30.7B total, 3.6B active) the base width.

The vertex of this parabola, $\log \eta^* = -b/2a$, yields the estimated optimal learning rate ($\eta^* = \exp(-b/2a)$) for the given token budget (B).

2.2.2 Extrapolating Optimal Learning Rates Across Long Token Horizons

We leverage the optimal learning rates estimated from short-token budget experiments via Equation (2) to extrapolate the optimal learning rate for the full target token horizon. Specifically, we perform a linear regression in log-log space between the estimated optimal learning rate (η^*) and the corresponding token budget (B):

$$\log(\eta^*) = \beta \cdot \log(B) + \gamma \quad (3)$$

where β and γ represent the regression coefficients. This regression model enables the extrapolation of the optimal learning rate to significantly larger token horizons (e.g., 10T tokens) without additional computational cost.

In experiments that employ batch size scheduling—increasing the batch size after a pre-determined number of steps—we restrict the regression to data points collected after the training dynamics have stabilized following the batch size increase. This ensures that the optimization regime is consistent across the fitted points and avoids artifacts arising from transient instabilities during the batch size transition.

3 Experiments and Analysis

3.1 Experimental Setup

We use the Muon optimizer (Jordan et al., 2024), which has been shown to accelerate convergence while maintaining competitive performance compared to conventional optimizers (e.g., AdamW (Loshchilov & Hutter, 2017)). The extended stable phase of the WSD scheduler (Hu et al., 2024) allows batch size scheduling to be applied effectively without disrupting training dynamics. Accordingly, batch size scheduling is applied in experiments using the WSD scheduler. Further details of the experimental setup are provided in Appendix A. All experiments are conducted on NVIDIA H200 GPUs, using an internal fork of Megatron-LM (Shoeybi et al., 2019).

3.2 μ P and Learning Rate Transfer with MoE Models

We set the base proxy MoE model’s hidden dimension (d_{model}) to 256, the total number of experts (n_{experts}) to 16, and the number of attention heads to 4. Following state-of-the-art large MoE models such as DeepSeek-V3 (Liu et al., 2024b) and Kimi-K2.5 (Team et al., 2026), we adopt Multi-head Latent Attention (MLA) (Liu et al., 2024a) for all models. MLA compresses the key-value cache into a low-dimensional latent space, significantly reducing KV cache memory overhead during inference and long-context extension. We then scale the model width by factors of $2\times$, $4\times$, and $8\times$, proportionally increasing the hidden dimension, total number of experts, and number of attention heads. As the total number of experts increases, the MoE routed scaling factor (Liu et al., 2024b) is adjusted for each model size, following the gate scaling factor calculation defined by Liu et al. (2025). Detailed model configurations are listed in Table 3 in Appendix B. All models are trained for 1.3B tokens from a general knowledge English corpus. For each model size, we sweep the maximum learning rate over $\{1, 3, 6\} \times 10^{-4}$, $\{1, 3, 4, 6\} \times 10^{-3}$, and $\{1, 3\} \times 10^{-2}$ to validate transferability.

Figure 3 demonstrates that width-scaled MoE models can successfully share optimal learning rates—defined as the learning rate yielding the lowest training loss—under μ P (Yang & Hu, 2021). In contrast, without the μ P learning rate described in Section 2.1 (referred to as Standard Parameterization, SP), the optimal learning rate fails to transfer across model widths. Corresponding results for dense models are included in Appendix C.

3.3 Finding Optimal Learning Rates Across Long Token Horizons

3.3.1 Learning Rate Dynamics Across Token Horizons

As Section 3.2 validates that μ P enables learning rate transfer across width-scaled MoE models, we further investigate how the optimal learning rate shifts as the training token budget increases. We define an MoE model with 5.6B total and 1.8B active parameters as the small-scale proxy model, and a $2\times$ width-scaled model with 20.7B total and 3.8B active parameters as the held-out target model. Both models employ μ P and are trained up to 100B tokens using a diverse data mixture that resembles realistic pretraining scenarios. Detailed model configurations are listed in Table 3.

Figure 4 visualizes quadratic fits (Eq. (2)) of validation loss against log-transformed learning rates (6×10^{-4} , $\{1, 3, 6\} \times 10^{-3}$). As the token budget increases, the optimal learning rate exhibits a slight downward trend. Notably, at each token scale, the fitted parabolas for both proxy and width-scaled target models exhibit highly consistent curvature and vertex locations, indicating that the optimal learning rate at a given token scale (i.e., the vertex of each parabola) transfers reliably across model widths.

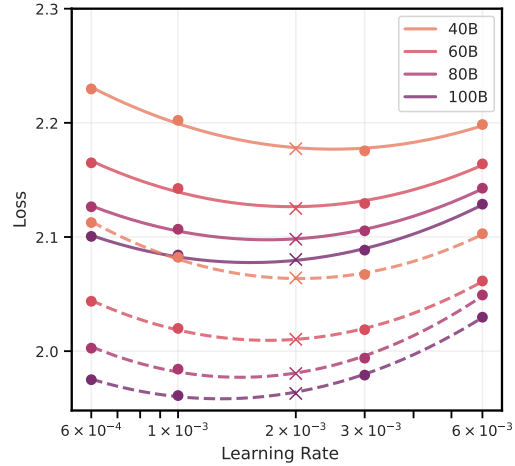


Figure 4: Solid lines (—) denote the base proxy model (5.6B total, 1.8B active), while dashed lines (---) represent the $2\times$ width-scaled held-out model (20.7B total, 3.8B active), each trained on 40B, 60B, 80B, and 100B tokens. The $\times$ markers indicate the actual validation losses at a learning rate of 2×10^{-3} , which fall directly on the quadratic curves fitted using the remaining four learning rates.

3.3.2 Predicting Optimal Learning Rate for Extremely Long Token Horizon

In realistic large-scale MoE training scenarios, models are often pretrained over extremely long token horizons (e.g., over 10T tokens). Accordingly, we extrapolate the findings from Section 3.3.1 to estimate the optimal learning rate for training at the 10T-token scale.

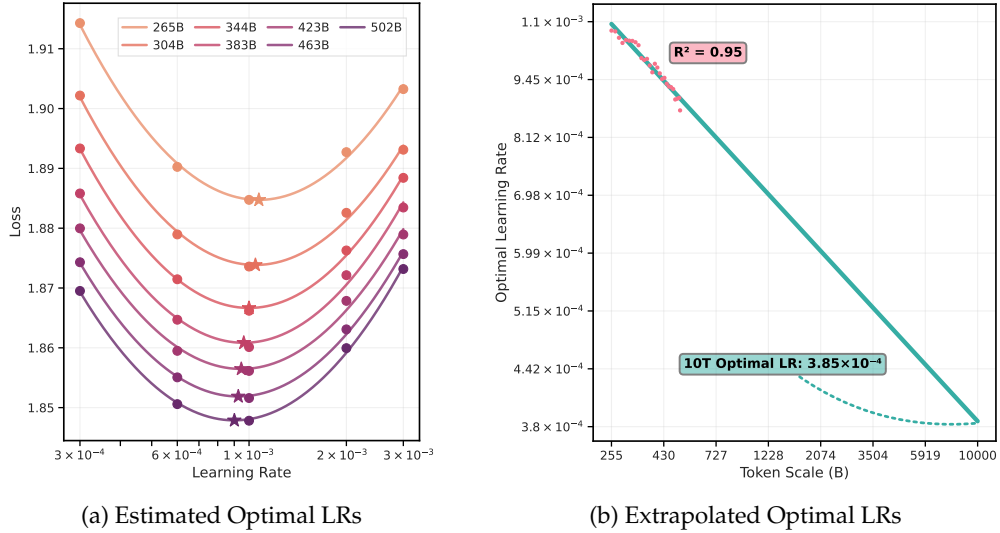


Figure 5: (a) Second-order polynomials fitted between the log-transformed learning rate and validation loss for the base proxy MoE model (10.8B total, 3.3B active) at representative token scales (a subset of all scales is shown for visual clarity). The $\star$ markers denote the optimal learning rates derived from the vertex of each parabola. (b) Linear regression in log-log space over these optimal learning rates across token scales ranging from 255B to 502B. The fitted model achieves $R^2 = 0.95$, enabling a reliable extrapolation of the optimal learning rate 3.85×10^{-4} for the 10T-token pretraining.

We first train the base proxy model (10.8B total, 3.3B active; corresponding to $1/4 \times$ width-scale of the target MoE model size, 155B total and 17B active parameters) for approximately 500B tokens. Both model configurations are listed in Table 3 in Appendix B. We then estimate the optimal learning rate at 10B-token intervals, utilizing EMA weights computed every 2B tokens. Figure 5a illustrates the estimated optimal learning rates—derived from the vertices of the fitted parabolas—across varying token scales.

Following the methodology in Section 2.2.2, we fit a linear regression model to extrapolate the optimal learning rate for larger token horizons. Specifically, we restrict the fitted data points to token budgets after 255B, ensuring that training dynamics have stabilized following the batch size increase. As shown in Figure 5b, the linear fit achieves a high R^2 score of 0.95, predicting an optimal learning rate of 3.85×10^{-4} for 10T-token training. This demonstrates that the optimal learning rate can be reliably inferred by extrapolating the linear regression from small proxy runs. Additional empirical support for this extrapolation is provided by the held-out validation in Appendix E.

3.3.3 Applying to Our Foundation Model

To validate the effectiveness of our approach, we apply the proposed two-step hyperparameter transfer framework to predict the optimal learning rate for pretraining a large-scale MoE foundation model (155B total, 17B active parameters) over a 10T-token horizon. Detailed model configurations are provided in Table 3. As shown in Figure 2b, the full-scale pretraining of this MoE model requires approximately $98 \times$ the total compute of the proxy runs used for optimal learning rate prediction.

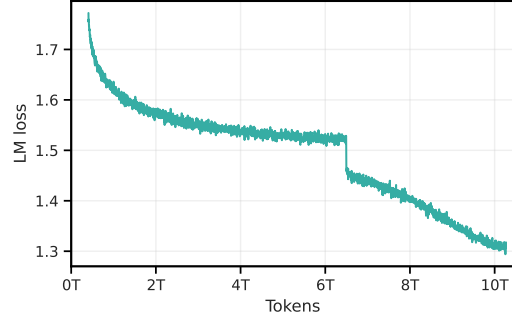


Figure 6: Training loss of our model (155B total, 17B active) during Stage 1 pretraining.

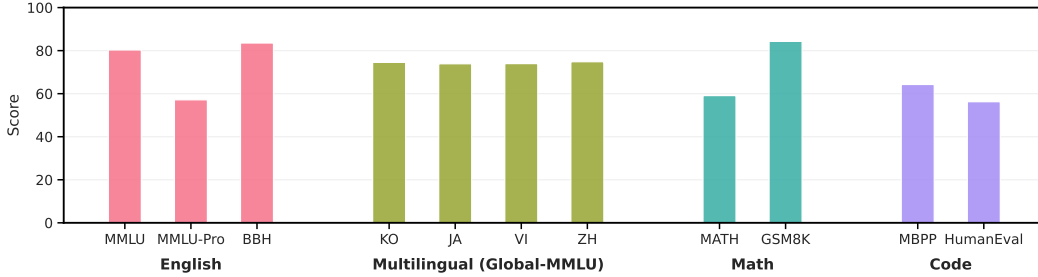


Figure 7: Benchmark performance of our foundation MoE model (155B total, 17B active) after Stage 1 pretraining on 10T tokens. Scores are presented across four categorized domains: English (MMLU, MMLU-Pro, BBH), Multilingual (Global-MMLU: Ko, Ja, Vi, Zh), Math (MATH, GSM8K), and Code (MBPP, HumanEval).

The Stage 1 pretraining data mixture for our model initially consists of 45% English, 12.5% Math/STEM, 27.5% Code, and 15.0% Multilingual corpora. At the 6T-token mark, we adjust the mixture to 22.5% English, 27.5% Math/STEM, 25.0% Code, and 25.0% Multilingual corpora to enhance coverage of underrepresented domains.

Figure 6 shows the training loss during Stage 1 pretraining, following batch size scheduling applied after 200B tokens. The pretraining loss of our foundation model remains highly stable throughout, with no loss spikes, confirming the validity of the extrapolated optimal learning rate. Evaluation results from Stage 1 pretraining are presented in Figure 7, with evaluation details available in Appendix D. Given that the Stage 1 data mixture prioritizes domain diversity over strict quality filtering, these evaluation results confirm that the model was trained successfully and efficiently.

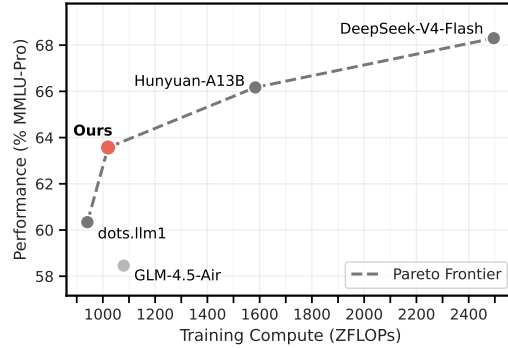


Figure 8: Estimated training compute vs. MMLU-Pro accuracy. Compute is estimated as $6ND$ with N set to active parameters.

In addition, figure 8 compares our model with publicly available open-weight MoE base models of comparable scale and active parameter count: dots.llm1 (Huo et al., 2025), GLM-4.5-Air (Zeng et al., 2025), Hunyuan-A13B (Tencent Hunyuan Team, 2025), and DeepSeek-V4-Flash (Xu et al., 2026). Training compute is estimated using the standard $6ND$ approximation (Kaplan et al., 2020; Hoffmann et al., 2022), with N set to active parameters and D to the reported pretraining token count. For consistency, all models are evaluated using our evaluation harness. Figure 8 shows that our model lies on the Pareto frontier, achieving higher MMLU-Pro (Wang et al., 2024b) accuracy than dots.llm1 and GLM-4.5-Air at comparable or lower estimated training compute.

4 Related Works

Hyperparameter Transfer As the scale of foundation models expands, the computational cost of finding optimal hyperparameters becomes increasingly prohibitive. To mitigate this, recent studies such as Maximal Update Parameterization (μP) (Yang & Hu, 2021) and μ -Transfer (Yang et al., 2022) enable zero-shot transfer of optimal hyperparameters (e.g., learning rate) across model width by modifying parameterization schemes and scaling learning rates or logits for specific tensors. Building on this, Wortsman et al. (2024) and Everett et al. (2024) have further proposed and verified a simplified adaptation of μP . While μP was initially applied to AdamW (Loshchilov & Hutter, 2017), it has recently been successfully extended to the Muon optimizer (Shah et al., 2025). However, existing studies that explore μP scaling for MoE models are mostly restricted to limited scenarios. For

example, Małaśnicki et al. (2025) attempt to apply μ P to the Switch Transformer (Fedus et al., 2022) architecture, but their study was limited to the AdamW optimizer (Loshchilov & Hutter, 2017) under restrictive settings, such as fixing the total and active expert counts and scaling only the hidden dimensions. These assumptions are difficult to generalize to large-scale MoE architectures that employ a large number of fine-grained experts (e.g., over 128), leaving reliable hyperparameter transfer for such powerful MoE models an open problem.

Scaling Laws As model size and training data grow, exhaustive hyperparameter sweeps become increasingly impractical, motivating the study of scaling laws to predict the optimal configurations. For MoE models, existing scaling research has largely focused on identifying optimal architectural configurations as the parameter count scales (Clark et al., 2022; Ludziejewski et al., 2024). Beyond model size, the training token horizon also significantly influences optimization dynamics, causing optimal hyperparameter configurations (e.g., learning rate, batch size) to shift. Recent studies have explored hyperparameter optimization based on computational budgets (Bi et al., 2024) and proposed extrapolating optimal configurations from small-scale proxies by jointly considering model and token scales (Bjorck et al., 2025; Li et al., 2025b). However, this joint scaling approach relies on sweeps across multiple dimensions (as depicted in Figures 1a and 2a).

5 Conclusion

We propose a compute-efficient two-step framework for hyperparameter transfer in large-scale Mixture-of-Experts (MoE) pretraining, enabling accurate optimal learning rate estimation without exhaustive sweeps. By combining μ P-based width transferability with a linear scaling law across token budgets, we show that optimal learning rates for trillion-token large-scale MoE pretraining can be reliably predicted from small proxy experiments, thereby reducing unnecessary computation. We validate this methodology by pretraining our foundation model (155B total, 17B active parameters) from scratch. While conducting exhaustive full-scale sweeps to definitively verify the optimality of the predicted learning rate (3.85×10^{-4}) is computationally infeasible, the exceptionally stable loss trajectory (Figure 6) and highly competitive benchmark scores (Figure 7) provide strong empirical evidence that our extrapolated configuration is effective and reliable for extreme-scale pretraining.

In this study, we focus on MoE architectures with Multi-head Latent Attention (MLA) (Liu et al., 2024a) and the Muon optimizer (Jordan et al., 2024). Extending this framework to diverse MoE structures and other optimizers remains an important direction for future work. Beyond architectural and optimizer generalization, we identify two additional research directions. First, because top- k routing can result in varying numbers of tokens assigned to individual experts, the effective batch size, gradient noise scale, and potentially the optimal update scale may vary across experts, suggesting that per-expert learning rate adaptation could yield further gains. However, realizing this in practice requires accounting for how routing evolves during training, how data mixture recipes affect expert-level batches, and how such adaptation can be implemented efficiently at scale. Given the substantial experimental and engineering costs these considerations entail, we leave per-expert learning rate adaptation to future work. Second, our large-scale recipe expands sparsity jointly with width axes rather than in isolation. Although our results demonstrate successful hyperparameter transfer along this joint scaling path, the effect of the sparsity dimension itself cannot be clearly disentangled from that of width. A controlled large-scale study of μ P transfer along the sparsity axis alone would therefore be valuable. We anticipate that extending the core principles—width-based μ P transfer and token-dimension scaling laws—to a broader range of settings will further reduce the computational burden of large-scale MoE research.

References

Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, et al. Phi-4

- technical report. *arXiv preprint arXiv:2412.08905*, 2024.
- Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, et al. Smollm2: When smol goes big—data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.
- Mikel Artetxe, Shruti Bhosale, Naman Goyal, Todor Mihaylov, Myle Ott, Sam Shleifer, Xi Victoria Lin, Jingfei Du, Srinivasan Iyer, Ramakanth Pasunuru, et al. Efficient large scale language modeling with mixtures of experts. *arXiv preprint arXiv:2112.10684*, 2021.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Baidu-ERNIE-Team. Ernie 4.5 technical report, 2025.
- Yoshua Bengio. Practical recommendations for gradient-based training of deep architectures. In *Neural networks: Tricks of the trade: Second edition*, pp. 437–478. Springer, 2012.
- Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiusi Du, Zhe Fu, et al. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024.
- Johan Bjorck, Alon Benhaim, Vishrav Chaudhary, Furu Wei, and Xia Song. Scaling optimal LR across token horizons. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=WYL4eFLcxG>.
- Cody Blakeney, Mansheej Paul, Brett W Larsen, Sean Owen, and Jonathan Frankle. Does your data spark joy? performance gains from domain upsampling at the end of training. *arXiv preprint arXiv:2406.03476*, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways, 2022. URL <https://arxiv.org/abs/2204.02311>, 1, 2022.
- Aidan Clark, Diego de Las Casas, Aurelia Guy, Arthur Mensch, Michela Paganini, Jordan Hoffmann, Bogdan Damoc, Blake Hechtman, Trevor Cai, Sebastian Borgeaud, et al. Unified scaling laws for routed language models. In *International conference on machine learning*, pp. 4057–4086. PMLR, 2022.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A Alemi, Roman Novak, Peter J Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, et al. Scaling exponents across parameterizations and optimizers. *arXiv preprint arXiv:2407.05872*, 2024.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.

- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL <https://openreview.net/forum?id=7Bywt2mQsCe>.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.
- Bi Huo, Bin Tu, Cheng Qin, Da Zheng, Debing Zhang, Dongjie Zhang, En Li, Fu Guo, Jian Yao, Jie Lou, et al. dots. llm1 technical report. *arXiv preprint arXiv:2506.05767*, 2025.
- Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 35:3843–3857, 2022.
- Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. Minimax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313*, 2025a.
- Houyi Li, Wenzhen Zheng, Qiufeng Wang, Hanshan Zhang, Zili Wang, Shijie Xuyang, Yuantao Fan, Zhenyu Ding, Haoying Wang, Ning Ding, et al. Predictable scale: Part i, step law—optimal hyperparameter scaling law in large language model pretraining. *arXiv preprint arXiv:2503.04715*, 2025b.
- Yunshui Li, Yiyuan Ma, Shen Yan, Chaoyi Zhang, Jing Liu, Jianqiao Lu, Ziwen Xu, Mengzhao Chen, Minrui Wang, Shiyi Zhan, et al. Model merging in pre-training of large language models. *Advances in Neural Information Processing Systems*, 38:133668–133691, 2026.
- Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024a.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024b.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.
- Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

- Jan Ludziejewski, Jakub Krajewski, Kamil Adamczewski, Maciej Pióro, Michał Krutul, Szymon Antoniak, Kamil Ciebiera, Krystian Król, Tomasz Odrzygóźdź, Piotr Sankowski, Marek Cygan, and Sebastian Jaszczur. Scaling laws for fine-grained mixture of experts. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=yoqdl ynCRs>.
- Jan Małaśnicki, Kamil Ciebiera, Mateusz Boruń, Maciej Pióro, Jan Ludziejewski, Maciej Stefaniak, Michał Krutul, Sebastian Jaszczur, Marek Cygan, Kamil Adamczewski, and Jakub Krajewski. Mu-parametrization for mixture of experts. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*, 2025. URL <https://openreview.net/forum?id=NK1jYM8BV0>.
- Daniel Morales-Brotons, Thijs Vogels, and Hadrien Hendriks. Exponential moving average of weights in deep learning: Dynamics and benefits. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=2M9CUnYnBA>.
- Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pp. 1–15, 2021.
- Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heine-man, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, et al. Olmo 3. *arXiv preprint arXiv:2512.13961*, 2025.
- Ishaan Shah, Anthony M Polloreno, Karl Stratos, Philip Monk, Adarsh Chaluvaraju, Andrew Hojel, Andrew Ma, Anil Thomas, Ashish Tanwer, Darsh J Shah, et al. Practical efficiency of muon for pretraining. *arXiv preprint arXiv:2505.02222*, 2025.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Shivalika Singh, Angelika Romanou, Clémentine Fourier, David Ifeoluwa Adelani, Jian Gang Ngui, Daniel Vila-Suero, Peerat Limkonchotiwat, Kelly Marchisio, Wei Qi Leong, Yosephine Susanto, et al. Global mmlu: Understanding and addressing cultural and linguistic biases in multilingual evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 18761–18799, 2025.
- Leslie N Smith. A disciplined approach to neural network hyper-parameters: Part 1–learning rate, batch size, momentum, and weight decay. *arXiv preprint arXiv:1803.09820*, 2018.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, 2023.
- Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- Tencent Hunyuan Team. Hunyuan-a13b, 2025. URL <https://github.com/Tencent-Hunyuan/Hunyuan-A13B>.

- Changxin Tian, Kunlong Chen, Jia Liu, Ziqi Liu, Zhiqiang Zhang, and Jun Zhou. Towards greater leverage: Scaling laws for efficient mixture-of-experts language models. *arXiv preprint arXiv:2507.17702*, 2025a.
- Changxin Tian, Jiapeng Wang, Qian Zhao, Kunlong Chen, Jia Liu, Ziqi Liu, Jiaxin Mao, Wayne Xin Zhao, Zhiqiang Zhang, and Jun Zhou. Wsm: decay-free learning rate schedule via checkpoint merging for llm pre-training. *arXiv preprint arXiv:2507.17634*, 2025b.
- Evan Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, et al. 2 olmo 2 furious (colm’s version). In *Second Conference on Language Modeling*, 2025.
- Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*, 2024a.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024b.
- Mitchell Wortsman, Peter Liu, Lechao Xiao, Katie Everett, Alexander Alemi, Ben Adlam, John D Co-Reyes, Izzeddin Gur, Abhishek Kumar, Roman Novak, et al. Small-scale proxies for large-scale transformer training instabilities. In *International Conference on Learning Representations*, volume 2024, pp. 49844–49869, 2024.
- Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, et al. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Greg Yang and Edward J. Hu. Tensor programs iv: Feature learning in infinite-width neural networks. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 11727–11737. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/yang21c.html>.
- Greg Yang, Edward J Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. *arXiv preprint arXiv:2203.03466*, 2022.
- Greg Yang, James B Simon, and Jeremy Bernstein. A spectral condition for feature learning. *arXiv preprint arXiv:2310.17813*, 2023.
- Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, et al. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.
- Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chengxing Xie, Cunxiang Wang, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
- Hanlin Zhang, Depen Morwani, Nikhil Vyas, Jingfeng Wu, Difan Zou, Udaya Ghai, Dean Foster, and Sham Kakade. How does critical batch size scale in pre-training? *arXiv preprint arXiv:2410.21676*, 2024.

Model Scale	Total Params.	Active Params.	n_{layers}	d_{model}	d_{ff}	n_{heads}	n_{experts}	λ_{routed}
Models for Section 3.2 and Appendix C								
Dense Base proxy	0.24B	0.24B	48	384	1536	4	-	-
Dense 2 $\times$ scale	0.7B	0.7B	48	768	3072	8	-	-
Dense 4 $\times$ scale	2.27B	2.27B	48	1536	6144	16	-	-
Dense 8 $\times$ scale	8.02B	8.02B	48	3072	12288	32	-	-
MoE Base proxy	0.6B	0.3B	48	256	6144	4	16	2.428
MoE 2 $\times$ scale	2.2B	0.7B	48	512	6144	8	32	2.442
MoE 4 $\times$ scale	8B	1.5B	48	1024	6144	16	64	2.446
MoE 8 $\times$ scale	30.7B	3.6B	48	2048	6144	32	128	2.448
Models for Section 3.3.1								
MoE Base proxy	5.6B	1.8B	32	1024	11264	16	32	2.442
MoE 2 $\times$ scale	20.7B	3.8B	32	2048	11264	32	64	2.446
Models for Figure 2, Section 3.3.2 and Section 3.3.3								
MoE Base proxy	10.8B	3.3B	62	1024	12288	16	32	2.442
MoE 1.5 $\times$ scale	23B	4.9B	62	1536	12288	24	48	2.445
MoE 2 $\times$ scale	40.2B	6.9B	62	2048	12288	32	64	2.446
MoE 4 $\times$ scale	155B	17B	62	4096	12288	64	128	2.448

Table 3: Model configurations for scaling width. n_{layers} , d_{model} , d_{ff} , and n_{heads} represent the number of layers, hidden dimension, intermediate dimension of an MLP (active experts for MoE; $d_{\text{ff}} = k \times d_{\text{expert}}$), and attention heads, respectively. n_{experts} is the total number of experts, and λ_{routed} denotes the MoE routed scaling factor.

A Training Details

We match Muon optimizer’s update RMS to that of AdamW by scaling the learning rate by $0.2 \cdot \sqrt{\max(A, B)}$, where A and B are the dimensions of the full-rank matrix parameter (i.e., fan_in and fan_out), following Liu et al. (2025). This adjustment allows for the direct transfer and reuse of the tuned learning rate and weight decay between the AdamW and Muon optimizers. We use weight decay with a coefficient of 1×10^{-1} and the auxiliary z-loss (Chowdhery et al., 2022) with a coefficient of 5×10^{-6} , both fixed as constants throughout all experiments. For validating μP for MoE with small-scale runs in Section 3.2, we employ the cosine learning rate scheduler (Loshchilov & Hutter, 2016) with a fixed minimum learning rate of 1×10^{-5} , varying only the maximum learning rate following Wortsman et al. (2024) and Li et al. (2025b). All other experiments use WSD scheduler (Hu et al., 2024).

We use a fixed batch size of 0.5M tokens for experiments in Section 3.2 and 8M tokens for Section 3.3.1. For Section 3.3.2, we increase the batch size from 8M tokens to 32M tokens after training on 200B tokens.

B Model Configuration Details

Table 3 details the dense and MoE model configurations for both the base proxy model and width-scaled models. All models employ Multi-head Latent Attention (MLA) (Liu et al., 2024a) for the attention mechanism. Model depth (i.e., the number of layers) is kept constant across all configurations. For the MoE models, we keep both the intermediate dimension of the expert MLP layers (d_{expert}) and the number of active experts (k) fixed during width-scaling. As a result, $d_{\text{ff}} = k \times d_{\text{expert}}$ remains unchanged. Furthermore, for the experiments in Section 3.3.2 and Section 3.3.3, we replace the first layer of the MoE model with a single dense layer whose intermediate dimension equals the activated intermediate dimension of the MoE layers (i.e., $k \times d_{\text{expert}}$).

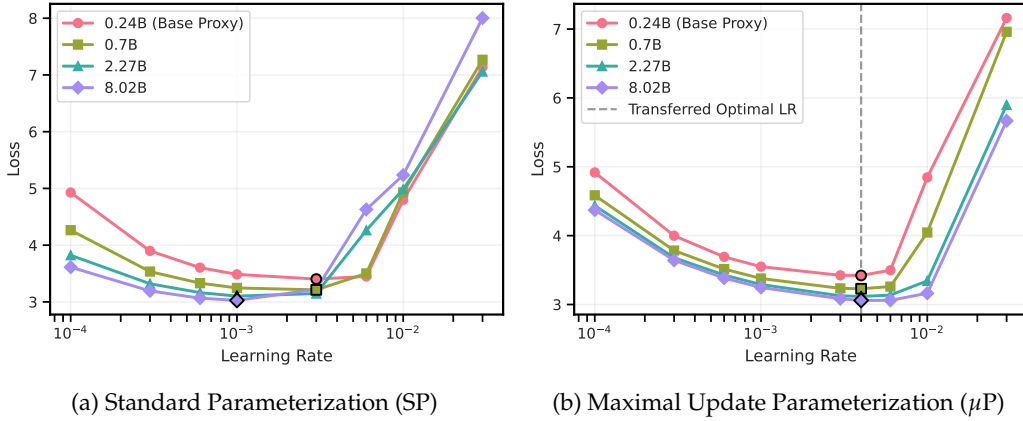


Figure 9: Comparison of learning rate transferability under Standard Parameterization (SP) and Maximal Update Parameterization (μP) across dense models of increasing width. Markers with black outlines indicate the learning rate achieving the lowest training loss (i.e., optimal learning rate) for each model. (a) Under SP, the optimal learning rate diverges across varying model widths. (b) Under μP , the optimal learning rate identified in the base proxy (0.24B) consistently transfers across models scaled to $2\times$ (0.7B), $4\times$ (2.27B), and $8\times$ (8.02B) the base width, demonstrating robust zero-shot hyperparameter transfer in dense MLA models as well as MoE models.

C Learning Rate Transfer with Dense Models

Since the combination of Multi-head Latent Attention (MLA) (Liu et al., 2024a) and the Muon optimizer (Jordan et al., 2024) has not been extensively explored under the μP and learning rate scaling protocols, we first validate the efficacy of hyperparameter transfer on a dense model baseline before extending the analysis to Mixture-of-Experts (MoE) architectures. For MLP layers in dense models, all fully connected layers are considered as the matrix-like hidden weights (Table 1).

We configure the base proxy model with a hidden dimension (d_{model}) of 384, an intermediate dimension (d_{ff}) of 1536 (four times the hidden dimension), and 4 attention heads. We then scale the model width to $d_{\text{model}} \in \{768, 1536, 3072\}$, d_{ff} to $\{3072, 6144, 12288\}$, and the number of attention heads to $\{8, 16, 32\}$, corresponding to parameter counts of 0.24B (base proxy), 0.7B, 2.27B, and 8.02B. All models have 48 layers and are trained for 1.3B tokens. For each model size, we sweep the maximum learning rate over $\{1, 3, 6\} \times 10^{-4}$, $\{1, 3, 6\} \times 10^{-3}$, and $\{1, 3\} \times 10^{-2}$ to validate transferability. Figure 9 demonstrates that under μP , the optimal learning rate that minimizes training loss is preserved across width-scaled Dense MLA models.

D Evaluation Details

For evaluating our model after Stage 1 pretraining (10T tokens), we use a customized evaluation framework based on Gao et al. (2024). Benchmarks are categorized as follows:

English General Knowledge: MMLU (5-shots) (Hendrycks et al., 2020), MMLU-Pro (5-shots) (Wang et al., 2024b), BBH (3-shots) (Suzgun et al., 2023)

Multilingual General Knowledge: Global-MMLU (Korean, Japanese, Vietnamese, Chinese; 5-shots) (Singh et al., 2025)

Math: MATH (4-shots) (Lewkowycz et al., 2022), GSM8K (8-shots) (Cobbe et al., 2021)

Code: MBPP (Pass@1, 3-shots) (Austin et al., 2021), HumanEval (Pass@1, 0-shot) (Chen et al., 2021)

E Small-Scale Validation of Learning Rate Extrapolation

Our two-step framework (Section 2) predicts the optimal learning rate (LR) at a long token horizon from a short-budget LR sweep. A direct validation at the 10T-token target is computationally infeasible: even for a proxy model at one-tenth the size of the target model (155B total, 17B active parameters), sweeping five learning rates for 10T tokens would consume roughly half the compute of the full 155B model training. Therefore, we perform a retrospective held-out validation using the LR sweep data from Section 3.3.2.

Specifically, among the token-budget data points in the 255B–500B range, we use only the first 11 budgets (approximately 255B–350B) to fit the extrapolation model. Following the same procedure as in the main paper, we first estimate the optimal learning rate at each budget using Eq. (2), and then fit the log-linear regression in Eq. (3). The resulting model is used to predict the optimal learning rates for five unseen token budgets near 500B, which are then compared against the optimal learning rates obtained by independently fitting Eq. (2) to the actual LR and loss values at each held-out budget.

Token Budget	Predicted Optimal LR	Actual Optimal LR	Ratio
462.7B	9.57×10^{-4}	9.26×10^{-4}	1.033×
472.6B	9.53×10^{-4}	9.22×10^{-4}	1.034×
482.5B	9.49×10^{-4}	8.97×10^{-4}	1.058×
492.4B	9.45×10^{-4}	8.99×10^{-4}	1.052×
502.3B	9.41×10^{-4}	9.01×10^{-4}	1.045×

Table 4: Held-out validation of learning rate extrapolation. The extrapolation model is fitted using only token budgets in 255B–350B and evaluated on unseen budgets near 500B. Ratios refer to Predicted Optimal LR divided by Actual Optimal LR.

As shown in Table 4, the predicted optimal learning rates (Predicted Optimal LR) closely match the independently estimated optima using Eq. (2) (Actual Optimal LR), with an average discrepancy of $\approx 4.4\%$. The gap is substantially smaller than the extrapolation errors reported by Bjorck et al. (2025). These results provide evidence that the proposed token-budget extrapolation accurately identifies the near-optimal learning-rate region over the range relevant to our two-step scaling procedure.

F Additional Analysis of Expert Routing Bias in Staged Pretraining

Having successfully completed Stage 1 pretraining of our foundation model (Section 3.3.3), we subsequently conducted Stage 2 training on a smaller-scale, high-quality dataset. During this process, we observed an interesting phenomenon related to expert routing dynamics that we share as an additional finding.

Staged pretraining—typically consisting of a large-scale general-domain training (Stage 1) followed by smaller-scale, high-quality, and domain-augmented training (Stage 2)—has been shown to be effective for training large language models from scratch (Hu et al., 2024; Blakeney et al., 2024; Abdin et al., 2024; Walsh et al., 2025; Allal et al., 2025). However, the resulting shift in data distribution may introduce instability in expert routing for MoE models.

As we adopt the auxiliary-loss-free load balancing strategy (Liu et al., 2024b), expert routing bias and the sequence-level load balancing loss are the primary factors stabilizing expert routing dynamics. Since sequence-level load balancing loss operates independently on each sequence irrespective of the underlying data distribution, we focus exclusively on factors related to expert bias. Specifically, we consider three expert bias configurations during Stage 2 pretraining: (1) initializing the expert bias with values obtained at the end of Stage 1 and

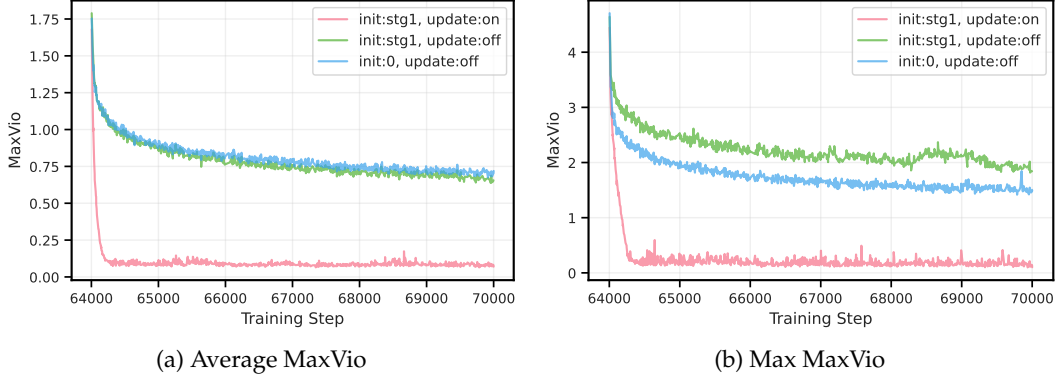


Figure 10: MaxVio trends for three different expert routing bias settings during Stage 2 training. (a) Average MaxVio across MoE layers. (b) Maximum MaxVio among all MoE layers at each training step.

continuing updates during Stage 2; (2) initializing with Stage 1 values but freezing updates during Stage 2; and (3) removing the expert bias terms entirely (initializing to zero and disabling updates).

Figure 10 shows notable differences in MaxVio (Wang et al., 2024a), a metric that reflects expert utilization imbalance:

$$\text{MaxVio} = \frac{\max_i \text{Load}_i - \overline{\text{Load}_i}}{\overline{\text{Load}_i}} \quad (4)$$

where Load_i denotes the number of tokens routed to the i -th expert, and $\overline{\text{Load}_i}$ stands for the expected number of tokens routed under perfect load balancing. MaxVio is computed at the global-batch level, and we report both the average and maximum values across MoE layers.

Our results indicate that continuously updating the expert bias during Stage 2 leads to the most balanced expert utilization. Interestingly, when expert bias updates are frozen, zero initialization is more beneficial for balanced expert utilization than inheriting values from Stage 1. This suggests that expert bias terms optimized for Stage 1 data distribution may not be suitable for the shifted distribution in Stage 2. Nevertheless, Figure 11 shows nearly identical training loss trajectories across all three configurations. This indicates that expert bias terms have a negligible impact on overall model optimization, implying that severe expert routing collapse is unlikely during Stage 2 regardless of the bias configurations.

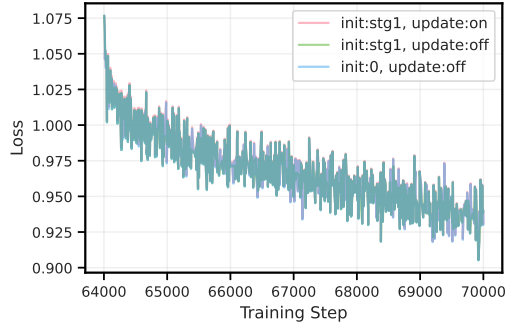


Figure 11: All three expert routing bias settings exhibit nearly identical training loss trends.

G Layer-wise Routing Balance and Expert Specialization

To further examine whether the relatively small MaxVio values of our foundation model (Section 3.3.3) during Stage 2 training (Figure 10) indicate overly balanced routing, we perform a layer-wise analysis of routing behavior. We evaluate routing on 200 prompts per benchmark (except for HumanEval, where all 164 available problems are used), drawn from four domain families: *English* (MMLU (Hendrycks et al., 2020), MMLU-Pro (Wang et al., 2024b), BBH (Suzgun et al., 2023)), *Multilingual* (Global-MMLU (Singh et al., 2025):

Ko, Ja, Vi, Zh), *Math* (MATH (Hendrycks et al., 2021), GSM8K (Cobbe et al., 2021)), and *Code* (MBPP (Austin et al., 2021), HumanEval (Chen et al., 2021)). For the *Code* domain, each prompt is concatenated with its reference solution so that routing is measured on tokens that include actual code. Benchmarks within the English, Math, and Code families are pooled together for measuring expert specialization, whereas each Global-MMLU language is treated separately.

We report three complementary metrics measuring marginal load imbalance and conditional expert specialization. They are defined as follows.

(i) Aggregate MaxVio. We use the same MaxVio definition as in Appendix F (Eq. (4)), computed on the whole evaluation set pooled across all target benchmarks. We refer to this as the Aggregate MaxVio. Lower Aggregate MaxVio indicates more balanced routing across experts.

(ii) Normalized mutual information $I(E; D)/H(D)$. Let $\mathcal{D}$ denote the set of domains, and let E and D denote the random variables for the selected expert and the domain label, respectively. For each domain $d \in \mathcal{D}$, let $p(e | d)$ be the conditional routing distribution over experts. We adopt a uniform domain prior $p(d) = 1/|\mathcal{D}|$, independent of per-domain token counts, so that larger domains do not dominate the statistic. Under this prior, the domain entropy is $H(D) = \log |\mathcal{D}|$, and the marginal routing distribution is $\bar{p}(e) = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} p(e | d)$. The mutual information between expert selection and the domain label is

$$I(E; D) = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} D_{\text{KL}}(p(e | d) \| \bar{p}(e)) \quad (5)$$

We normalize it by $H(D)$,

$$I/H = \frac{I(E; D)}{H(D)} = \frac{I(E; D)}{\log |\mathcal{D}|} \in [0, 1] \quad (6)$$

$I/H = 0$ means every domain uses experts with the same distribution, whereas $I/H = 1$ means expert selection perfectly identifies the domain. Intermediate values quantify the fraction of domain entropy resolved by expert routing—e.g., $I/H = 0.15$ indicates that routing resolves about 15% of the domain entropy and should not be interpreted as domain-classification accuracy. Higher values indicate stronger domain-dependent expert specialization.

(iii) Mean pairwise Jensen–Shannon divergence. For any two domains a and b with routing distributions $p(e | a)$ and $p(e | b)$, let $m = \frac{1}{2}(p(e | a) + p(e | b))$. Then

$$\text{JSD}(a, b) = \frac{1}{2} D_{\text{KL}}(p(e | a) \| m) + \frac{1}{2} D_{\text{KL}}(p(e | b) \| m) \quad (7)$$

The reported mean pairwise JSD is the average of $\text{JSD}(a, b)$ over all domain pairs. Using natural logarithms, the mean pairwise JSD is bounded above by $\log 2 \approx 0.693$. A mean pairwise JSD value of 0 means every pair of domains has identical expert routing distributions. Higher values indicate more distinct expert routing patterns across different domains.

Figure 12 shows that marginal expert load balance and conditional specialization behave differently across depth. Aggregate MaxVio remains relatively stable throughout most MoE layers, with only a localized increase in the shallow-to-middle layers. In contrast, both conditional specialization measures (I/H , mean pairwise JSD) increase toward deeper layers and reach their highest values in the final MoE layers. This demonstrates that balanced marginal routing does not imply weak expert specialization: experts can maintain balanced overall utilization while simultaneously exhibiting domain-dependent routing preferences. Note that the Aggregate MaxVio values here are computed on downstream evaluation benchmarks and are therefore not directly comparable to the training-time MaxVio in Figure 10; these analyses should be interpreted as complementary evidence of layer-wise routing behavior and expert specialization, rather than a direct numerical comparison.

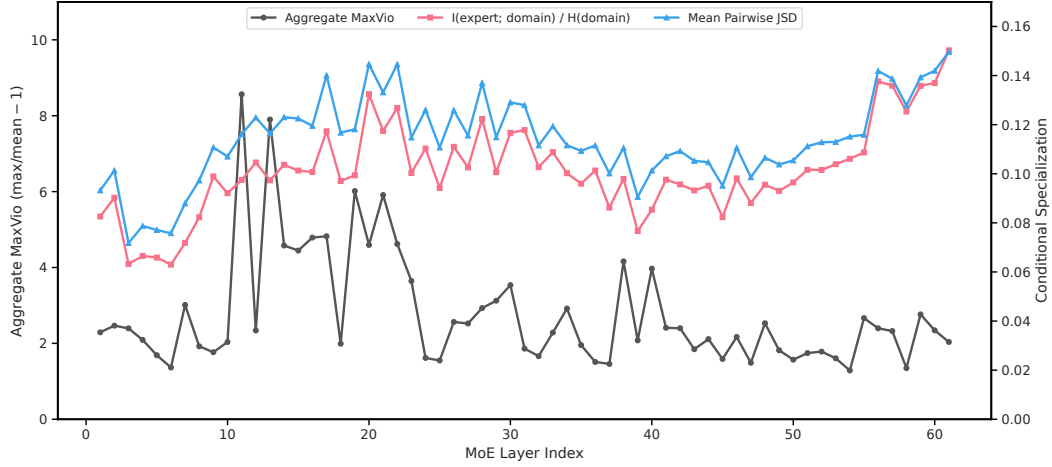


Figure 12: Aggregate MaxVio (left axis) measures marginal expert-load imbalance, while normalized mutual information (I/H) and mean pairwise Jensen–Shannon divergence (right axis) measure domain-dependent routing specialization. Specialization increases toward deeper MoE layers, whereas MaxVio follows a different trend, demonstrating that balanced marginal routing does not necessarily imply suppressed specialization.

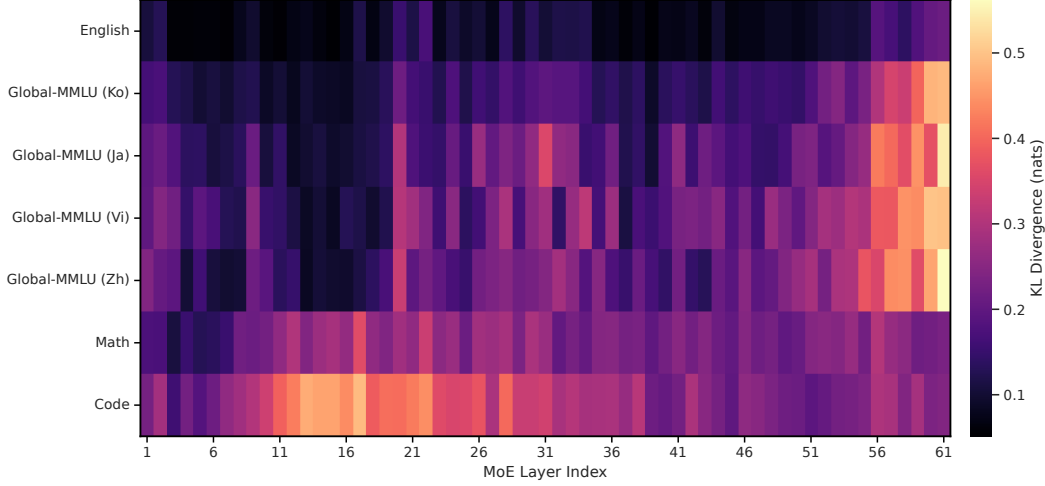


Figure 13: Domain-conditioned expert routing divergence across MoE layers. Each cell shows $D_{\text{KL}}(p(e | d) \| \bar{p}(e))$, where $\bar{p}(e)$ is the layer-wise marginal routing distribution defined in (ii). Brighter colors indicate stronger domain-specific routing.

To further illustrate the depth dependence of specialization, Figure 13 visualizes the KL divergence $D_{\text{KL}}(p(e | d) \| \bar{p}(e))$, where $p(e | d)$ denotes the expert routing distribution conditioned on domain d and $\bar{p}(e)$ is the marginal routing distribution aggregated across domains at the same layer. Larger values indicate stronger domain-specific routing relative to the shared marginal distribution.

The heatmap (Figure 13) reveals distinct specialization patterns across domains. Code routing deviates from the marginal distribution in the early layers, whereas multilingual routing remains close to the marginal until the final MoE layers, where specialization increases sharply. Together with Figure 12, these results show that balanced marginal routing can coexist with domain-dependent expert specialization.